# Structure as Semantics: Impedance Modeling and Retrenchment in Cyber-Physical Systems

John Baugh[1], Richard Banach[2(✉)], and Eunsuk Kang[3]

[1] North Carolina State University, Raleigh, NC 27695, USA
jwb@ncsu.edu
[2] The University of Manchester, Manchester M13 9PL, UK
richard.banach@manchester.ac.uk
[3] Carnegie Mellon University, Pittsburgh, PA 15213, USA
eunsukk@andrew.cmu.edu

**Abstract.** We address a gap in CPS verification: current methods focus on temporal behavior but give little support for reasoning about *physical structure* and its interface guarantees. We render structure as first class by using *impedance* as the interface semantics, so models from different formalisms can be compared on equal terms and design steps judged by their boundary effect. This algebraic view suits state-based tools, enabling reasoning about physics without solving differential equations. We mechanize the approach in Alloy, deriving composition theorems for series-parallel networks and recovering standard circuit laws. A case study shows how impedance-based contracts support stepwise design via *retrenchment*, with bounded approximations that preserve boundary behavior. The method applies across physical domains, providing a uniform interface semantics for CPS models and tools.

**Keywords:** Cyber-physical systems · impedance modeling · interface semantics · retrenchment · refinement · Alloy · formal verification

## 1 Introduction

Cyber-Physical Systems (CPS) represent an increasingly important class of systems in engineering and computing, with applications spanning transportation, infrastructure, robotics, and manufacturing [4,7,36,38]. These systems combine discrete computational logic with continuous physical processes. Ensuring their correctness requires methods that support reasoning not only about software but also about physics and their interaction [1,15,35].

Accounting for this interaction has typically proceeded in one of two ways. One is the use of hybrid automata or similar frameworks in which continuous behavior, governed by differential equations, is punctuated by discrete transitions. This offers a unified behavioral semantics at the level of traces and reachability, and connects well with verification tools. The other is to discretize that


A. Raschke et al. (Eds.): Egon Börger Festschrift, LNCS 16580, pp. 249–272, 2026.
https://doi.org/10.1007/978-3-032-24494-9_12

behavior entirely and reason within a purely symbolic or state-based formalism, where verification techniques are well developed and tooling is mature.

Both approaches are useful, but neither treats *physical structure* as a first-class semantic concern. Differential equations capture behavior with greater precision, yet offer little guidance for principled modeling, transformation, or reuse; they are downstream of those structural choices. Engineering workflows, by contrast, rely on transforming models—abstracting, refining, or reconfiguring components—while preserving the properties that matter. This raises a basic question: what counts as a valid transformation, and how should we characterize the equivalence classes it respects, for example, equality of external behavior at system interfaces?

These questions are difficult to answer with trace semantics alone. Distinct internal structures can give rise to the same external behavior, and many internal differences may not matter from the standpoint of interaction with the environment. To address this, we suggest a shift in perspective: to regard physical structure—defined by interconnections and constitutive relationships—as a form of semantics.

We formalize this perspective by taking **impedance** as the interface semantics and **retrenchment** as the mechanism for controlled deviation. Impedance enables comparison of models that differ internally yet interact with their environment in the same way. Structural transformations are evaluated by their effect on the impedance, bounded by a *quantified* deviation. Retrenchment provides the language to quantify and constrain that deviation, turning design approximations into statements with bounds (e.g., parameter intervals, admissible shifts in poles and zeros, or frequency-window tolerances).

This combination is versatile: (i) it is not tied to a particular formalism, so it supports reasoning across series-parallel networks [2,22], linear graphs [24, 37,39], bond graphs [21,31,32], and other interconnection schemes [25,41]; and (ii) it is compatible with mature state-based formalisms such as ASM, Alloy, B, TLA$^+$, VDM, and Z [33] that support reasoning over standard numerical domains. In control-theoretic terms, it mirrors the move from time-domain differential operators to algebraic transfer functions in the complex $s$-plane, via the Laplace transform, where differentiation and integration become multiplication and division by functions of $s$.

Our aim in this paper is to show how impedance modeling and retrenchment, taken together, provide a foundation for reasoning about physical structure and behavior in CPS. We work with structure as semantics, using impedance to compare models, and use retrenchment to make approximations explicit and bounded. The goal is a simple formal discipline: transformations have stated conditions; preservation means the same impedance on a declared domain; deviations are recorded as quantitative tolerances. This places common engineering steps inside a precise semantics, in the spirit of principled, semantically grounded modeling to which Egon Börger's work on Abstract State Machines has contributed so deeply [20].

*Paper Organization.* Section 2 introduces the structural impedance semantics that treats interconnection and constitutive laws as the meaning at an interface. Section 3 presents a finite, state-based mechanization in Alloy and derives composition results for series-parallel networks. Section 4 formulates bounded-deviation steps via retrenchment, with illustrative tolerance and component library substitution examples. Section 5 reports on checks and cross-validation against symbolic and numerical calculations, and discusses limitations. Section 6 situates the approach within CPS modeling and verification. Section 7 summarizes contributions and outlines directions for extending structure-as-semantics across richer interconnection formalisms.

## 2 Structural Impedance Semantics

This section develops the central idea that *impedance* can serve as a semantics for physical structure, a meaning that allows different models to be compared on equal terms, independent of how they are realized internally. We begin with the effort-flow view of external behavior, then define impedance-based equivalence and the algebraic laws it obeys. The approach is illustrated on a simple series RC network whose impedance provides the canonical example.

### 2.1 Interfaces and Effort-Flow Behavior

Every physical component interacts with its environment through an *interface* carrying two complementary variables: an *effort* (such as voltage, force, or pressure) and a *flow* (such as current, velocity, or volumetric flux). Their product is power. The observable behavior of a system is the relation between these quantities at its ports.

Figure 1 illustrates the single-port interface convention used here: a system driven by an ideal source at a boundary port that carries the pair $(E_{\text{in}}, F_{\text{in}})$. We follow the common across/through terminology, treating effort as the across variable and flow as the through variable; the instantaneous power into the system is $E_{\text{in}}F_{\text{in}}$.

The source shown is purely conceptual, representing any admissible boundary condition; our concern is with the system's response relation, not with the realization of the driving source itself.

For linear, time-invariant components this relation is linear:

$$e = z * f$$

in the time domain, or, under Laplace transform and zero stored energy,

$$E(s) = Z(s)\,F(s).$$

Here $Z(s)$ is the *impedance* and $Y(s) = 1/Z(s)$ its admittance. Either can serve as the model's external *meaning*: the function that relates boundary effort and flow. Internal equations of motion (ODEs, DAEs, or state realizations) are one way to realize the same external relation; they are not part of the semantic layer.

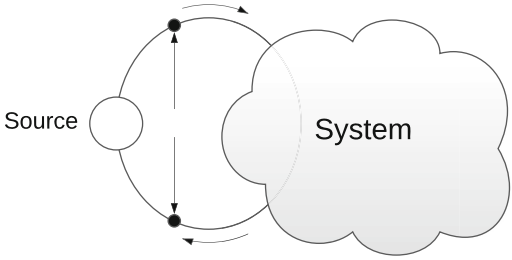


**Fig. 1.** Single-port interface view. An ideal source drives a system through a boundary port labeled with effort and flow.

### 2.2 Equivalence via Impedance

Two models are *equivalent* when they induce the same effort-flow relation at their boundary over a declared analysis domain $S$. We compare models extensionally, not by internal states. This permits cross-formalism reasoning: a series-parallel network, a linear graph, or a bond graph can all be considered equivalent if they exhibit the same mapping $E(s) \mapsto F(s)$.

For concreteness, we mechanize the series-parallel class of two-terminal networks used in our examples.

### 2.3 Syntax of Structures

Let $\mathsf{Net}$ be generated by the grammar

$$N ::= R(r) \mid L(\ell) \mid C(c) \mid N \bullet N \mid N \parallel N,$$

where $r, \ell, c \in \mathbb{R}_{>0}$. The operators $\bullet$ and $\parallel$ denote series and parallel composition, respectively.

### 2.4 Semantic Domain and Impedance Map

Let $F$ be a field containing $\mathbb{R}$ (specialized later to $F = \mathbb{C}$), and let $S \subseteq F$ be an *admissible domain* on which all denominators below are nonzero; for steady-state sinusoidal analysis we take $S = \{\, j\omega : \omega > 0 \,\}$. The impedance of $N \in \mathsf{Net}$ is the function $[\![N]\!] : S \to F$ defined recursively by

$$\begin{aligned}
[\![R(r)]\!](s) &= r, \\
[\![L(\ell)]\!](s) &= \ell\, s, \\
[\![C(c)]\!](s) &= \frac{1}{c\, s}, \\
[\![A \bullet B]\!](s) &= [\![A]\!](s) + [\![B]\!](s), \\
[\![A \parallel B]\!](s) &= \frac{[\![A]\!](s)\, [\![B]\!](s)}{[\![A]\!](s) + [\![B]\!](s)}.
\end{aligned}$$

Side conditions on $S$ ensure all denominators in the parallel rule are nonzero.

### 2.5 Equivalence (Preservation Obligation)

For $A, B \in \mathsf{Net}$ and domain $S$,

$$A \equiv_S \mathrm{B} \iff \forall s \in S : \ [\![A]\!](s) = [\![B]\!](s).$$

A structure-preserving rewrite is valid on $S$ if it maps $A$ to $B$ with $A \equiv_S B$. In this sense, algebraic laws such as series or parallel commutativity are verified by equality of impedances on the declared guard.

### 2.6 Basic Transformation Laws

For all $A, B, C \in \mathsf{Net}$ and all $s \in S$:

$$\begin{aligned} \text{Series commutativity: } & [\![A \bullet B]\!] = [\![B \bullet A]\!]. \\ \text{Series associativity: } & [\![(A \bullet B) \bullet C]\!] = [\![A \bullet (B \bullet C)]\!]. \\ \text{Parallel commutativity: } & [\![A \parallel B]\!] = [\![B \parallel A]\!]. \\ \text{Parallel associativity: } & [\![(A \parallel B) \parallel C]\!] = [\![A \parallel (B \parallel C)]\!], \end{aligned}$$

all interpreted where denominators are nonzero on $S$. These serve as the canonical *structure-preserving transformations*.

### 2.7 Running Example: Series *RC*

A resistor $R$ in series with a capacitor $C$, shown in Fig. 2, has impedance

$$Z(s) \ = \ R + \frac{1}{sC}.$$

Restricting to $s = j\omega$ with $1/j = -j$ and $\omega > 0$ gives

$$Z(j\omega) \ = \ R - \frac{j}{\omega C}.$$

The guard $S = \{j\omega : \omega > 0\}$ excludes the singularity at $\omega = 0$, where the capacitor is open-circuited. The external behavior is therefore completely determined by the pair $(R, C)$. Recalling that we restrict to steady-state sinusoidal analysis, for which $S = \{\, j\omega : \omega > 0 \,\}$, exact preservation of an arbitrary model $A$'s boundary behavior by another model $B$ means $[\![A]\!](j\omega) = [\![B]\!](j\omega)$ for all $\omega > 0$.

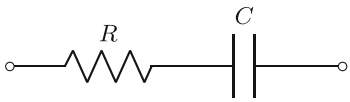


**Fig. 2.** Series $RC$ between two terminals corresponding to $R \bullet C$.

This simple case already captures the structure of more elaborate examples. The real and imaginary parts correspond to dissipative and reactive behavior; their combination forms the external impedance seen at the port. Later sections introduce tolerance and interval variants of $R$ and $C$, leading to bounded enclosures for $Z(j\omega)$ and quantitative *retrenchment* clauses.

### 2.8 What the Semantics is (and is Not)

By *structural impedance semantics* we mean a congruent, compositional interface semantics that maps a port-typed interconnection (the structure) to its boundary impedance—possibly multiport, frequency-dependent, or interval-valued—using series/parallel composition laws and context closure, so that design and proof proceed entirely by reasoning on the boundary.

The semantic object compared is the external effort-flow mapping $Z$, not an internal differential equation. Internal dynamics are merely one realization that induces the same $Z$. This makes the semantics compatible with state-based formalisms that support reasoning over numerical domains. Transformations appear as guarded rewrites whose preservation conditions reduce to algebraic equalities over the reals.

In short, the semantics tells us exactly what aspects of a physical model we are declaring to be significant: its interface behavior, not its internal realization. This matters because CPS modeling tools vary widely in how they describe continuous dynamics, but they all agree on the boundary quantities that govern interaction. The impedance semantics gives a common algebraic target—one that survives abstraction, structural rewriting, discretization, or even translation into a finite-state setting.

What the semantics does *not* do is replace differential equations or provide a new physical theory. It simply identifies the interface quantity that is preserved (or intentionally relaxed) by a transformation, and turns that into a concrete obligation. In this sense, it plays the same role that trace semantics plays for software refinement, but for physical structure: it supplies the glue that lets disparate descriptions be compared and composed.

### 2.9 Cross-Domain Analogy

The same structural semantics appears in many energy domains once effort and flow variables are reinterpreted (cf. the *analog computer* viewpoint of Feynman et al. [27]). For instance, in mechanics effort is force and flow is velocity; the analog of resistance is damping, and the analog of capacitance is compliance (spring constant). A pair of springs in series satisfies

$$\frac{1}{k_{\text{eq}}} = \frac{1}{k_1} + \frac{1}{k_2},$$

which is the same algebra as two capacitors in series. Such correspondences make the impedance semantics portable across physical domains and provide a unifying algebraic view of structure.

## 3 Alloy Modeling of Structural Semantics

The denotational view of Sect. 2 provides an algebraic meaning for networks: each structure denotes an impedance function $[\![N]\!] : S \to F$ that maps frequency

to a complex value. To make this semantics executable and inspectable, we encode it in Alloy, a lightweight modeling language with a SAT-based analyzer that explores finite instances within a user-declared scope [30]. Alloy provides a declarative relational logic with built-in integers, and its Analyzer reduces assertions to SAT, automatically producing satisfying instances or counterexamples.

### 3.1 Alloy as Executable Approximation

Before turning to the model, it is important to distinguish two levels of semantics. The real-valued impedance semantics of Sect. 2 is the *ideal* domain in which preservation and equivalence are defined. Alloy provides a *finite approximation* of that semantics: numeric expressions are interpreted over bounded integers, and division at capacitive leaves uses the Analyzer's floor semantics. For algebraic obligations (series/parallel laws, commutativity, associativity), this approximation is exact, because no division is required in the check. For impedance bounds involving capacitors, we enforce sound, division-free inequalities, ensuring that every Alloy instance faithfully under-approximates its real-valued counterpart.

To make this concrete, we encode the series–parallel syntax and its impedance equations as relational constraints over these bounded integers. Within this setting, Alloy can check equivalence obligations, generate counterexamples, and, as a byproduct, perform *structural synthesis*.

The analysis is not a numerical simulation but a structural check over all configurations within a chosen scope. The *small-scope hypothesis* [6,29] suggests that most structural flaws have small witnesses, and the SAT-based engine can search the corresponding space effectively. This makes structural impedance semantics mechanically inspectable, rather than purely paper-based.

### 3.2 Basic Syntax

Here, we briefly introduce the basic syntax of Alloy. A more complete language description can be found in [30].

The Alloy keyword **`sig`** introduces a *signature*, which defines a set of elements in the universe. A signature may contain one or more *fields*, each introducing a relation that maps the elements of the signature to the field expression.

The **`extends`** keyword creates a subtyping relationship; an **`abstract`** signature has no elements except those belonging to its extensions, and **`one sig`** introduces a signature that contains only one element.

An Alloy **`fact`** is a set of constraints that holds for every satisfying instance of the model. A *predicate* (**`pred`**) is a parameterized encapsulation of constraints that can be reused across the model. A *function* (**`fun`**) returns an expression given zero or more parameters.

### 3.3 Core Signatures and Tree Structure

Returning to structural semantics, the Alloy model mirrors the abstract grammar of networks defined in Sect. 2. Each network node carries an impedance $Z$ of

type `Complex`; internal nodes connect subtrees; leaves carry component parameters. The `tree` constraint ensures acyclicity, enforcing the intended series-parallel structure rather than arbitrary graphs.

**Listing 1.1.** Core Alloy signatures.

```
// A complex number has real and imaginary parts
sig Complex {
  re, im: one Int
}

// Every network node has an impedance Z
abstract sig Node {
  Z: one Complex
}

// Binary operator nodes: the network is a tree
abstract sig Binary extends Node {
  left, right: one Node
}
{ left != right }

fun children: Node→Node { left + right }
fact { tree[children] }

// Internal nodes: series or parallel
sig Series, Parallel extends Binary {}

// Leaf nodes: components with a coefficient
abstract sig Component extends Node {
  val: one Int
}
{ val > 0 }

sig R, L, C extends Component {}
```

Figure 3 illustrates the corresponding metamodel automatically rendered by the Alloy Analyzer. Series and parallel nodes are binary; all nodes are linked to a single `Complex` impedance object. This schema constitutes the structural part of the semantics.

### 3.4 Leaf Laws and Propagation to $Z$

Each component imposes a simple algebraic form on its impedance: resistors contribute real positive terms, inductors positive imaginary terms, and capacitors negative imaginary terms. We evaluate on the imaginary axis ($s = j\omega$): a purely oscillatory input yields a complex impedance, with $Z$.re capturing the resistive part and $Z$.im the reactive part.

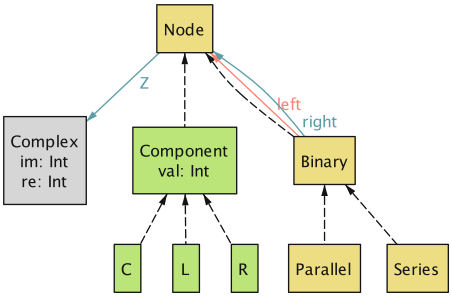


**Fig. 3.** A metamodel of the core Alloy signatures. Each dotted edge represents a subset relationship (i.e., **extends**), and a labeled edge represents a user-defined relation.

**Listing 1.2.** Leaf shapes on the imaginary axis ($s = j\omega$).

```
fact LeafShapes {
  all r: R | Z[r].im = 0 and Z[r].re > 0          // resistor
  all l: L | Z[l].re = 0 and Z[l].im > 0          // inductor
  all c: C | Z[c].re = 0 and Z[c].im < 0          // capacitor
}
```

The capacitor requires special handling because its impedance involves a reciprocal. To reduce explicit division and keep most arithmetic structural, we store its value as the inverse capacitance $D = 1/C$, so $1/(sC)$ becomes $D/s$. Consequently, each capacitor's numeric field `c.val` represents $D$ rather than $C$, and the imaginary component of its impedance is computed as $-D/\omega$.

**Listing 1.3.** Leaf values.

```
// Global parameter: analysis frequency
one sig Params {
  omega: one Int
}
{ omega > 0 }

// Leaf values (numerical law)
fact LeafValues {
  all r: R | Z[r].re = r.val
  all l: L | Z[l].im = mul[Params.omega, l.val]
  all c: C | Z[c].im = negate[div[c.val, Params.omega]]
}
```

Series and parallel combinations follow the usual algebraic rules. In composition we avoid division by expressing the parallel rule via the product identity $Z\,(Z_a + Z_b) = Z_a\,Z_b$; the only divisions occur in leaf evaluation.

**Listing 1.4.** Series and parallel combination rules.

```
fact SeriesRule {
  all n: Series |
```

```
    some rhs: Complex |
      add[Z[n.left], Z[n.right], rhs] and eq[Z[n], rhs]
}

fact ParallelRule {
  all n: Parallel |
    some _sum, lhs, rhs: Complex |
      add[Z[n.left], Z[n.right], _sum] and
      mul[Z[n], _sum, lhs] and
      mul[Z[n.left], Z[n.right], rhs] and
      eq[lhs, rhs]
}
```

These facts constitute a literal mechanization of the mapping $[\![\cdot]\!]$ from Sect. 2. The representation is faithful for all points where denominators of the rational laws are nonzero. With the structural and numerical rules in place, we can now express the algebraic preservation laws as Alloy assertions.

### 3.5 Transformation Obligations as Assertions

Structure-preserving transformations become Alloy **assert** statements. For example, the commutativity of series composition is encoded as follows.

**Listing 1.5.** Representative transformation check.

```
assert SeriesComm {
  all a,b: Node |
    some s1, s2: Series |
      left[s1]=a and right[s1]=b and
      left[s2]=b and right[s2]=a implies
      Z[s1] = Z[s2]
}
check SeriesComm for 5 but 7 Complex, 6 Node, 6 Int
```

The **check** command tells Alloy to search for a counterexample to `SeriesComm` within a finite, user-given scope. For example, `for 5 but 7 Complex, 6 Node, 6 Int` means: explore all models whose signatures contain at most 5 atoms (except for `Complex`, `Node`, and `Int`, which are capped at 7, 6, and 6 atoms respectively). Alloy then performs a bounded but exhaustive search over this space. A successful check, indicated by the absence of any counterexample, would imply that the commutativity obligation holds for all structures within the explored scope. On the other hand, a counterexample instance, if it exists, would describe a particular structural configuration where equality fails.

Before working through larger examples, it is useful to clarify the status of Alloy's integer arithmetic and the role of explicit guards.

### 3.6 Guards, Partiality, and Arithmetic Soundness

Because the impedance algebra involves partial operations (e.g., division by $\omega$), guard conditions must be stated explicitly in the formalization. In Alloy,

these guards are written directly as constraints on each analysis state. Since Alloy's arithmetic is bounded and integral, all proofs and counterexamples occur within a finite domain of small integers, and every analysis state is a sampled frequency with integer-valued parameters.

Within this guarded domain, the integer model should be read as a finite, coarse sampling of the intended real-valued semantics. Any difference Alloy finds corresponds to a real semantic difference at the sampled frequency: if two structures yield distinct impedances under the bounded integer interpretation, then their real-valued impedances also differ at that state. The converse does *not* hold. Integer truncation and finite sampling can collapse nearby real-valued behaviors, so Alloy may report two structures as indistinguishable even though they differ outside the chosen scope or below the truncation granularity.

In this sense, the arithmetic is a sound *detector of differences* but not a completeness guarantee for equivalence. Section 4 later extends this finite analysis by introducing bounded intervals that summarize entire parameter ranges and make the loss of precision explicit in the form of quantified budgets using retrenchment.

### 3.7 Example: RLC Series Circuit

We model Example 3.4 from Ch. 10 of Chua et al. [22], which analyzes a standard series $RLC$ network in the Laplace domain. In our notation,

$$N \;=\; R(6) \;\bullet\; L(1) \;\bullet\; C(25),$$

where $R$, $L$, and $C$ denote the resistor, inductor, and capacitor (represented by its inverse parameter $D = 1/C$). Evaluated on the imaginary axis, the impedance function has the form

$$Z(j\omega) \;=\; R + j\big(\omega L - D/\omega\big),$$

whose imaginary part crosses zero at $\omega_0 = \sqrt{D/L} = 5$, the expected resonance frequency.

The Alloy fragment below declares this two-level series topology and, within a finite scope for $\omega$, symbolically computes the same impedance relation.

**Listing 1.6.** RLC series circuit (Chua example).

```
pred RLC_topology [r: R, l: L, c: C, s1, s2: Series] {
  left[s1] = r and right[s1] = l
  left[s2] = s1 and right[s2] = c
  R = r and L = l and C = c
  Series = s1 + s2 and no Parallel
}
pred RLC_series [w: Int] {
  Params.omega = w
  some r: R, l: L, c: C, s1, s2: Series |
    RLC_topology[r, l, c, s1, s2] and
```

```
    r.val = 6 and l.val = 1 and c.val = 25
}
run RLC_series for 3 but 7 Complex, 5 Node, 6 Int
```

When the above **run** command is executed, the Analyzer reproduces the expected qualitative pattern of the impedance (constant real part and sign change in the imaginary component at $\omega_0 = 3$) confirming that the encoded semantics faithfully captures the analytic behavior; the tool further visualizes the generated instance as a graph diagram to aid in comprehension (Fig. 4).

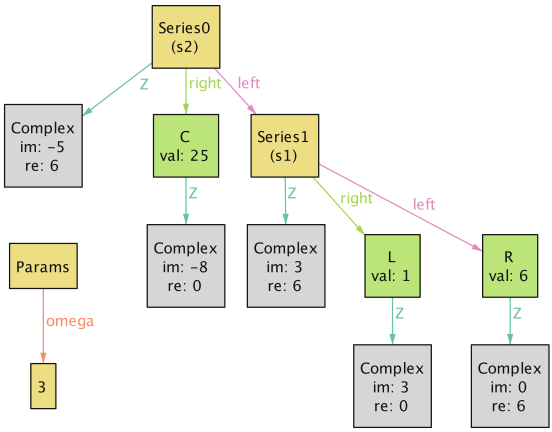


**Fig. 4.** A visualization of an instance representing the RLC series circuit with $\omega_0 = 3$.

### 3.8 Synthesis by Impedance Box Specification

The same encoding supports synthesis by treating the impedance box as a specification. The predicate `inBox` defines rectangular bounds in the complex plane.

**Listing 1.7.** Specification of a target impedance box.

```
pred inBox [c: Complex, re_lo, re_hi, im_lo, im_hi: Int] {
  c.re ≥ re_lo and c.re ≤ re_hi
  c.im ≥ im_lo and c.im ≤ im_hi
}

pred synth {
  some n: Node | Params.omega = 5 and inBox[Z[n], 6, 6, 0, 0]
}
run synth for 3 but 6 Complex, 5 Node, 7 Int
```

Given the above **run** command, Alloy enumerates candidate network topologies (up to the specified scope) to generate an instance whose impedance evaluates to 6 when $\omega = 5$. These amount to: a single R(6); a parallel pair followed by

a series resistor $(R_1 \parallel R_2) \bullet R_3$; and other configurations that are algebraically equivalent to these. The model checker thus doubles as a symbolic search engine for structure-preserving solutions.

### 3.9 Extending to Multiple Analysis States

So far $\omega$ has been fixed globally. To move beyond single-frequency evaluation, we introduce a small set of analysis states:

```
sig State { omega: Int } { omega > 0 }
```

Each impedance becomes a two-dimensional relation `Z[node, s]`. Unlike temporal traces, these states are independent frequency samples. A total order over `State` aids inspection in Alloy's instance viewer: one can "scroll" through frequency slices to observe how each box moves in the complex plane. Multi-$\omega$ analysis is the foundation for synthesis tasks based on desired frequency responses.

### 3.10 Synthesis by Frequency Behavior: Low-Pass Filter

Introducing `State` allows specifications to span multiple frequencies rather than a single operating point. Our goal is to synthesize a simple *low-pass* network directly from its frequency behavior. Instead of prescribing a topology, we constrain the impedance trend: at low frequency the imaginary part should be large (capacitive), at high frequency smaller (more resistive), while the real part remains roughly constant, thereby suppressing unwanted high-frequency content (e.g., noise and interference in radar systems).

The following predicates express this requirement over two analysis states, representing $\omega = 6$ and $\omega = 24$.

**Listing 1.8.** Low-pass synthesis by behavior over two frequencies.

```
pred RC_series [root: Series] {
  some r: R, c: C {
    left[root] = r
    right[root] = c
    r.val = 2
    c.val = 48      // recall: 48 = 1/C
    isRooted[root]
  }
}

pred isLowPass [n: Node, s0, s1: State, Z_re, Z_lo, Z_hi: Int] {
  inBox[Z[n].s0, Z_re, Z_re, min[Int], Z_lo]   // low freq: larger |Im|
  inBox[Z[n].s1, Z_re, Z_re, Z_hi, max[Int]]   // high freq: smaller |Im|
}

pred synthLowPass {
  some n: Node |
    let s0 = so/first, s1 = s0.next |
```

```
      no_strays[n] and
      s0.omega = 6 and s1.omega = 24 and
      isLowPass[n, s0, s1, 2, -7, -3]
}
run synthLowPass for 3 but 12 Complex, 5 Node, 7 Int, exactly 2 State
```

Alloy searches the space of admissible interconnections and finds a network whose impedance decreases in magnitude as frequency increases, consistent with low-pass behavior. One solution is the series composition $C(42) \bullet R(2)$, satisfying $Z(6) = 2 - j7$ and $Z(24) = 2 - j1$. No topology is assumed in advance; the impedance boxes alone suffice to characterize and synthesize the desired behavior. Constraining `#L = 0` rules out inductors and reduces search time.

### 3.11 Discussion and Limits

These examples show how impedance-based semantics can be executed in a state-based tool: Alloy checks structural equivalence, finds counterexamples, and synthesizes networks satisfying interface specifications. Within Alloy's bounded-integer semantics, the treatment is *structurally exact* but *numerically approximate*: Although counterexamples confirm the failure of the stated property over any domain including the domain that is tested, confirmation of the property over the tested domain can only *suggest*, but not guarantee, its validity over a larger domain.

Other mature state-based frameworks, such as B and Event-B (Rodin), ASM, and ProB, support reasoning over richer numeric domains through symbolic proof and SMT back-ends, although their power is often constrained somewhat by being targeted at transition systems executing over real time, or some abstraction of real time (such as a sequence of states in succession). Despite this, the approaches mentioned are sufficient for the algebraic reasoning required here, since the impedance semantics needs only field properties of $\mathbb{R}$ or $\mathbb{Q}$ over rational functions, and not limits or continuity. In that spectrum, Alloy provides a lightweight and executable member, emphasizing exploration and synthesis rather than absolute verification via symbolic proof.

The next section moves from exact identities to controlled approximations via *retrenchment*: per-frequency boxes, deviation bounds, and quantified interface guarantees.

## 4 Retrenchment: Bounded Deviation at the Interface

Exact preservation ($Z_A \equiv Z_B$) may be too strong. For example, component tolerances, operating-point linearization, and library rounding all introduce small, intentional mismatches. Rather than stating only the ideal, we attach an explicit, auditable *tolerance* at the interface. We capture such departures from the ideal with *retrenchment* [10,12–14].

Retrenchment, as described in the cited references, is intended for the real time (or abstraction of real time) state-based point of view. As such, it suffers

from a similar mismatch to the frequency domain, and need for a better fit to it, as do the state-based verification approaches mentioned above.

Briefly, retrenchment arose as an attempt to characterize deviation from exact preservation of a refinement (or gluing, or retrieve) relation between the state spaces of a step of a formal refinement of an abstract state-based transition model to a concrete one. The deviation was captured via a number of relations (as discussed in the next section). In the early days of experimentation with the concept, e.g. [12,13], these were combined in various ways in the main retrenchment proof obligation, in order to capture the deviation at issue. This line culminated in [14]. After some time, the concept was reappraised and simplified in [10], still for the time-oriented state-based point of view. None of these variants is ideal for the purpose here.

### 4.1 Clauses and Reading

Formulated in a way more suitable for the frequency domain needed in this paper, a retrenchment step from $A$ to $B$ over an admissible domain $S_{\text{guard}}$ is recorded with:

WITHIN : Preconditions and guards (frequency band, side conditions).
RETRIEVE : The ideal relation (the refinement target), typically $Z_A(s) = Z_B(s)$ for all $s \in S_{\text{guard}}$.
OUTPUT : What is actually compared at the interface (e.g. $Z_A(s)$ inside a tolerance set around $Z_B(s)$).
CONCEDES : A quantitative budget that bounds the deviation permitted by the step.

Exact preservation is the degenerate case OUTPUT $\equiv$ RETRIEVE and CONCEDES $\equiv$ OUTPUT. In approximate steps, CONCEDES is the engineering "ledger entry."

Unlike the time domain oriented state-based versions, there is no proof obligation that is required to hold featuring the above clauses in the present frequency domain adaptation of retrenchment. However, like the state-based versions, there are straightforward composition laws for successive retrenchment steps.

For the retrenchment formulations mentioned earlier, composition was studied extensively in [11]. Here though, because CONCEDES records facts about *any* acceptable OUTPUT, the way the composition laws work in the present formulation is a little different.

Thus if A, B, and C are systems, with a retrenchment from A to B, captured using $W_{\text{A,B}}$, $R_{\text{A,B}}$, $O_{\text{A,B}}$, $C_{\text{A,B}}$, (in an obvious notation), and a retrenchment from B to C, captured using $W_{\text{B,C}}$, $R_{\text{B,C}}$, $O_{\text{B,C}}$, $C_{\text{B,C}}$, then these can be composed to yield a retrenchment from A to C, captured using:

$$
\begin{aligned}
W_{\mathrm{A,C}} &\equiv (\exists \blacklozenge_B \bullet W_{\mathrm{A,B}} \wedge W_{\mathrm{B,C}}) \\
R_{\mathrm{A,C}} &\equiv (\exists \blacklozenge_B \bullet R_{\mathrm{A,B}} \wedge R_{\mathrm{B,C}}) \\
O_{\mathrm{A,C}} &\equiv (\exists \blacklozenge_B \bullet O_{\mathrm{A,B}} \wedge O_{\mathrm{B,C}}) \\
C_{\mathrm{A,C}} &\equiv (\exists \blacklozenge_B \bullet C_{\mathrm{A,B}} \wedge C_{\mathrm{B,C}})
\end{aligned}
$$

This differs from the formulations in [11] in that the law for $C_{\mathrm{A,C}}$ is purely conjunctive. Also, in the above, the '$\exists \blacklozenge_B$' refers to the existential quantification of any constants or variables that are relevant exclusively to B. The composition is evidently associative.

### 4.2 Two Illustrative Steps

*(i) Resistor Tolerance.* Replacing a nominal $R(r)$ by a manufactured part $R(r')$ with $r' \in [r(1-\delta), r(1+\delta)]$ is:

$$
\begin{aligned}
\textsc{Within} &: s \in S_{\mathrm{guard}},\ r > 0,\ \delta \in [0,1), \\
\textsc{Retrieve} &: r' = r, \\
\textsc{Output} &: Z_A(s) = r \in [\, r(1-\delta),\, r(1+\delta)\,] =: \widehat{Z}_B(s), \\
\textsc{Concedes} &: \mathrm{width}(\widehat{Z}_B) = 2r\delta \quad (\text{equivalently } |r - r'| \le r\delta).
\end{aligned}
$$

The impedance is real and independent of $s$, so the box is constant on $S_{\mathrm{guard}}$.

*(ii) Series of Two Capacitors (or Two Springs).*

With $[\![C(c)]\!](s) = \frac{1}{cs}$ and $c_{\mathrm{eq}}$ s.t. $\frac{1}{c_{\mathrm{eq}}} = \frac{1}{c_1} + \frac{1}{c_2}$,

$$
\textsc{Within}:\ s \in S_{\mathrm{guard}},\ s \neq 0,\ c_1, c_2 > 0, \qquad \textsc{Retrieve}:\ \frac{1}{c_1 s} + \frac{1}{c_2 s} = \frac{1}{c_{\mathrm{eq}} s}.
$$

If a single library element $\tilde{c}$ is chosen instead,

$$
\textsc{Output}:\ \left| \tfrac{1}{c_1 s} + \tfrac{1}{c_2 s} - \tfrac{1}{\tilde{c}\, s} \right| = \frac{\left| \frac{1}{c_1} + \frac{1}{c_2} - \frac{1}{\tilde{c}} \right|}{|s|}, \qquad \textsc{Concedes}:\ \frac{\left| \frac{1}{c_1} + \frac{1}{c_2} - \frac{1}{\tilde{c}} \right|}{|s|} \le \varepsilon(s).
$$

High frequencies (large $|s|$) tighten the bound; low frequencies loosen it.

### 4.3 Choosing the Guard $S_{\mathrm{guard}}$

The guard states where the algebraic semantics is well-defined and relevant:

AC band: $S_{\mathrm{guard}} = \{\, j\omega : \omega \in [\omega_{\min}, \omega_{\max}]\,\}$; Half-plane: $\Re(s) \ge 0$ minus poles;
Structural side-conditions: $s \neq 0$, $[\![A]\!] + [\![B]\!] \neq 0$ for parallel subterms.

Narrower guards usually admit tighter Concedes budgets.

### 4.4 Local Retrenchment via Interval Semantics

We model component uncertainty directly and push it to the interface. Let

$$r \in [r^-, r^+], \quad \ell \in [\ell^-, \ell^+], \quad c \in [c^-, c^+], \quad s \in S_{\text{guard}}.$$

Pointwise (in $s$) interval semantics at the boundary is:

$$[\![R([r^-, r^+])]\!] = [r^-, r^+], \quad [\![L([\ell^-, \ell^+])]\!] = s \cdot [\ell^-, \ell^+], \quad [\![C([c^-, c^+])]\!] = \Big[\frac{1}{c^+ s}, \frac{1}{c^- s}\Big].$$

A *retrieve* relation that glues a concrete interval model $Z_C^\sharp : S \to \mathcal{I}(\mathbb{C})$ to an abstract point model $Z_A : S \to \mathbb{C}$ is:

$$R(c, a) \ : \ \forall s \in S_{\text{guard}} : \ Z_A(s) \in Z_C^\sharp(s).$$

Retrenchment then asserts WITHIN $S_{\text{guard}}$ and records a budget, e.g. $\text{width}\big(Z_C^\sharp(s)\big) \le \varepsilon(s)$. Crucially, we perform *structural rewrites symbolically* and introduce intervals only at the interface, avoiding dependency blow-up inside the algebra.

### 4.5 Integer Semantics and Division-Free Bounds

Our Alloy mechanization uses integer arithmetic with a frequency parameter $\omega > 0$ in each `State`. Capacitor leaves implement

$$\Im Z_C(\omega) = -\Big\lfloor \frac{D}{\omega} \Big\rfloor, \quad \text{with } D = \frac{1}{C} \in \mathbb{Z}_{>0},$$

so *floor* semantics appears at the leaves. The tolerance information for a single RC configuration is packaged into a singleton record.

**Listing 1.9.** RC Box: tolerance parameters for a single RC configuration.

```
one sig RCBox {
  R0, Rmin, Rmax: Int,
  D0, DMin, DMax: Int     // (D = 1/C)
} {
  0 ≤ Rmin and Rmin ≤ R0 and R0 ≤ Rmax
  0 ≤ DMin and DMin ≤ D0 and D0 ≤ DMax
}
```

Mathematically, this corresponds to fixing a tolerance "RC box" with $\Re Z \in [R_{\min}, R_{\max}]$ and $D \in [D_{\min}, D_{\max}]$, and then deriving the corresponding bounds on $\Im Z$ for each $\omega > 0$.

We eliminate explicit division in membership tests via the standard property of floor:

$$\forall d \in \mathbb{Z}, \ \forall \omega > 0 : \qquad q = \lfloor d/\omega \rfloor \iff q\,\omega \le d < (q+1)\,\omega.$$

Because $d$ is integral, the strict upper bound is equivalent to

$$d \;\le\; (q+1)\omega - 1 \;=\; q\omega + (\omega - 1).$$

Thus, for a tolerance "RC box" with real and imaginary bounds

$$\Re Z \in [R_{\min}, R_{\max}], \qquad \Im Z \in \big[-\lfloor D_{\max}/\omega \rfloor,\, -\lfloor D_{\min}/\omega \rfloor\big],$$

the equivalent *division-free* test is

$$R_{\min} \le \Re Z \le R_{\max}, \qquad -D_{\max} \;\le\; \omega\, \Im Z \;\le\; -D_{\min} + (\omega - 1),$$

which is exact for a series $RC$ (and other linear sums) and avoids integer truncation mismatches.

In Alloy, we capture this as a predicate `inRCBox[z, s, box]`, parameterized by an explicit box.

**Listing 1.10.** Division-free membership test for the RC box.

```
pred inRCBox [z: Complex, s: State, box: RCBox] {
  s.omega > 0
  re[z] ≥ box.Rmin and re[z] ≤ box.Rmax
  mul[im[z], s.omega] ≥ negate[box.DMax]
  mul[im[z], s.omega] ≤ negate[box.DMin] + (s.omega - 1)
}
```

In subsequent RC examples we use the unique instance `RCBox`, with the understanding that the underlying bounds come from the fixed record `RCBox`.

### 4.6 Lightweight Alloy Obligations (RC Running Example)

We use small *safety* assertions rather than existential "witness" assertions, so scopes without components do not spuriously fail checks. Below are representative obligations; the core model defines `Within_RC`, `RC_nominal`, `RC_interval`, and the division-free `inRCBox` predicate described above.

**Listing 1.11.** Alloy safety obligations for nominal and interval RC configurations.

```
// Any nominal RC (R0,D0) lies inside its RCBox at every admissible state.
assert RC_NominalInside_AllStates {
  all s: State | Within_RC[s] implies
    all root: Series, r: R, c: C |
      RC_nominal[root, r, c] implies inRCBox[root.Z.s, s, RCBox]
}

// Nominal RC is inside the same RCBox at two ordered analysis states.
assert RC_Nominal_Safety_2States {
  let s0 = so/first, s1 = s0.next |
    Within_RC[s0] and Within_RC[s1] implies
      all root: Series, r: R, c: C |
        RC_nominal[root, r, c] implies
          inRCBox[root.Z.s0, s0, RCBox] and
```

```
          inRCBox[root.Z.s1, s1, RCBox]
}

// Any RC whose parameters lie in RCBox (interval case) also stays inside
// RCBox at both ordered analysis states.
assert RC_Interval_Safety_2States {
  let s0 = so/first, s1 = s0.next |
    Within_RC[s0] and Within_RC[s1] implies
      all root: Series, r: R, c: C |
        RC_interval[root, r, c] implies
          inRCBox[root.Z.s0, s0, RCBox] and
          inRCBox[root.Z.s1, s1, RCBox]
}
```

In practice, we run checks with `0 L` to eliminate inductors in RC-focused assertions. All three assertions pass under modest scopes (e.g., `for 3 but 12 Complex, 5 Node, 0 L, 7 Int, 2 State`).

*Widths as* Concedes *Budgets.* On each `State` with $\omega > 0$ the box widths are

$$w_{\Re} = R_{\max} - R_{\min}, \qquad w_{\Im} = \left\lfloor \frac{D_{\max}}{\omega} \right\rfloor - \left\lfloor \frac{D_{\min}}{\omega} \right\rfloor,$$

and these serve directly as per-frequency Concedes quantities.

Retrenchment localizes approximation to the interface: structure gives the algebra; the algebra yields $Z(\cdot)$; and any deviation is recorded as an explicit, frequency-dependent budget. The Alloy harness shows these obligations are small, composable, and compatible with integer semantics.

## 5 Evaluation and Discussion

*Symbolic and Numerical Cross-checks.* The impedance semantics and retrenchment pattern were mechanized in Alloy and cross-checked against symbolic and numerical approaches. Alloy successfully discharged preservation and bounded-deviation obligations for representative RC networks, confirming that the semantics can be realized in a finite, state-based setting. Independent spot checks in Isabelle/HOL [34] verified the corresponding algebraic identities, while simulations in Julia [16] at sampled frequencies reproduced the same boundary impedances, providing an empirical validation of the formal results. Together these demonstrate coherence across formal, symbolic, and numerical domains.

*Alternative Numeric Encodings.* Our Alloy model uses bounded integers with floor division, which keeps analysis lightweight and makes the impedance semantics executable without introducing new numeric theories. Other encodings are possible. One could introduce a rational type or a small field-like numeric structure, providing exact algebra for the impedance laws and eliminating truncation effects at capacitive leaves. Such encodings offer richer arithmetic at the cost of

increased solver time. Alloy and its SMT-backed extensions provide only integer arithmetic as a numeric theory; they do not support real arithmetic or expose bitvector or uninterpreted-function theories at the user level. Nonetheless, a type with field-like properties would better match the algebraic needs of impedance networks and could support a wider class of transformations and retrenchment steps.

*Dependency and Interval Limitations.* A continuing challenge is the treatment of dependency in interval arithmetic. The inversion strategy used here (computing pointwise numeric responses and enclosing them afterward) avoids the usual dependency blow-up but only for monotone, linear compositions. Nonmonotone or coupled systems will require richer enclosures (e.g., affine or symbolic) and tighter reasoning about shared uncertainty to preserve soundness under retrenchment.

*Linear Systems.* The perspective in this paper is formulated around Linear Time-Invariant (LTI) systems, reflecting the fact that CPS analysis and control are most often carried out on linearized models around operating points. Impedance semantics is naturally suited to this setting: series/parallel composition and frequency-domain reasoning are linear constructions, and transfer-function equivalence is expressed directly at the interface. Retrenchment then provides a way to record the gap between a nonlinear plant and its linear surrogate: the WITHIN clause delimits the operating region of the linearization, and the CONCEDES clause records the allowed deviation. Nonlinear dynamics and global behaviors are not treated here; the focus is on the LTI core that underpins most practical CPS workflows.

## 6 Related Work

A long line of work treats *structure* itself as a semantic concern. Börger's Abstract State Machines [19], algebraic specification, and categorical approaches to systems all highlight the semantic importance of structural relationships. In the physical domain, bond graphs and port-Hamiltonian models express this through conservation principles and interconnection rules that determine system behavior. Our use of impedance belongs to this tradition: it takes the boundary effort-flow relation as the semantic object and evaluates transformations by how they preserve (or intentionally relax) that interface behavior. A recent survey by Bliudze et al. [17] echoes this structural viewpoint, identifying CPS design as a sequence of modeling, discretization, and simulation steps rooted in physical interconnection structure. Their emphasis on structural equational models and linear/bond-graph semantics aligns closely with our motivation for providing a precise, algebraic interface semantics.

A second line of work concerns the integration of continuous and discrete dynamics. Hybrid automata [5,28], differential dynamic logic and KeYmaera X [35], Hybrid Event-B [8,9], set-based reachability analysis [3], and statistical model checkers such as UPPAAL SMC [23] provide expressive formalisms

for reasoning about trajectories, invariants, reachability, and switching. Our contribution is orthogonal: we do not analyze trajectories or flows, but provide a denotational interface model that can sit alongside such formalisms, supplying a structural equivalence and approximation relation that is independent of the internal dynamics.

Finally, engineering practice relies heavily on simulation-centric frameworks such as the Modelica language and standard library [26,40], the Functional Mock-up Interface (FMI) for co-simulation [18], and Simulink/Stateflow development tools by MathWorks Inc. These environments provide executable models of continuous-time behavior, but they do not by themselves offer structural guarantees across transformations or abstractions. Impedance-based semantics offers a complementary notion of correctness: it isolates the boundary quantity that must be preserved across refactoring, code generation, discretization, or component substitution, providing a lightweight alternative to trajectory-level validation.

## 7 Conclusions

This work rests on two simple observations about CPS modeling and verification:

1. Starting from differential equations undermines physical reasoning at the interface.
2. Impedance provides the interface semantics that all physical models must agree on.

These observations clarify the role of structural semantics in CPS. Differential equations describe behavior but do not, on their own, express the physical constraints that arise from interconnection, energy conservation, and component-level structure. Impedance recovers this missing interface layer and provides the common quantity that any physical description must satisfy. This makes it possible to reason about transformations, abstractions, and approximations in a way that is independent of the modeling formalism, yet grounded in the physics that governs system interaction.

*Why it Matters.* We demonstrate that state-based methods can reason about physical semantics directly, not by simulation but by algebraic structure. This opens the door for symbolic verification tools to engage with the physics of CPS design in a principled, semantics-driven way, bridging the gap between what engineers simulate and what formal methods can prove.

*Future Directions.* A natural next step is to generalize the structural impedance semantics beyond series-parallel networks to bond graphs and linear graphs, making explicit the correspondence between different interconnection schemes and showing how they preserve (or transform) impedance at the interface. Extending the approach in this way would help unify multiple CPS modeling

paradigms under a common algebraic semantics and test how structural equivalence principles scale to more complex networks.

Another direction is to move beyond steady-state sinusoidal response. The present work interprets $Z(s)$ on the imaginary axis $s = j\omega$, focusing on frequency-response behavior. More generally, impedance is given by an analytic function in the right half-plane where the Laplace transform (which underpins all such frequency domain approaches in the engineering world) is defined. Allowing the analysis domain $S$ to range over this half-plane, or other judiciously chosen portions of it, rather than just the imaginary axis, would connect the same structural semantics to transient behavior and initial conditions via the inverse transform. Developing that link to the time-domain view common in CPS verification remains an open direction for future work.

## References

1. Abrial, J.R., Su, W., Zhu, H.: Formalizing hybrid systems with Event-B. In: Abstract State Machines, Alloy, B, VDM, and Z - Third International Conference, ABZ 2012, Pisa, Italy, 18-21 June 2012. Proceedings. Lecture Notes in Computer Science, vol. 7316, pp. 178–193. Springer (2012)
2. Alexander, C.K., Sadiku, M.N.: Fundamentals of Electric Circuits. McGraw-Hill, 4th edn. (2009)
3. Althoff, M., Frehse, G., Girard, A.: Set propagation techniques for reachability analysis. Ann. Rev. Control Robot. Autonomous Syst. **4**(1), 369–395 (2021)
4. Alur, R.: Principles of Cyber-Physical Systems. MIT Press, Cambridge, MA (2015)
5. Alur, R., et al.: The algorithmic analysis of hybrid systems. Theoret. Comput. Sci. **138**(1), 3–34 (1995)
6. Andoni, A., Daniliuc, D., Khurshid, S.: Evaluating the "small scope hypothesis". Tech. Rep. MIT-LCS-TR-921, MIT CSAIL, Cambridge, MA, USA (2003)
7. Baheti, R., Gill, H.: Cyber-physical systems. In: Samad, T., Annaswamy, A. (eds.) The Impact of Control Technology: Overview, Success Stories, and Research Challenges. IEEE Control Systems Society (2011)
8. Banach, R., Butler, M., Qin, S., Verma, N., Zhu, H.: Core hybrid event-B I: single hybrid event-B machines. Sci. Comp. Prog. **105**, 92–123 (2015)
9. Banach, R., Butler, M., Qin, S., Zhu, H.: Core hybrid event-B II: multiple cooperating hybrid event-B machines. Sci. Comp. Prog. **139**, 1–35 (2017)
10. Banach, R.: Graded refinement, retrenchment, and simulation. ACM Trans. Softw. Eng. Methodol. **32**(2) (2023)
11. Banach, R., Jeske, C., Poppleton, M.: Composition mechanisms for retrenchment. J. Logic Algebraic Program. **75**, 209–229 (2008)
12. Banach, R., Poppleton, M.: Retrenchment: an engineering variation on refinement. In: International Conference of B Users, pp. 129–147. Springer (1998)
13. Banach, R., Poppleton, M.: Sharp retrenchment, modulated refinement and simulation. Formal Aspects Comput. **11**, 498–540 (1999)
14. Banach, R., Poppleton, M., Jeske, C., Stepney, S.: Engineering and theoretical underpinnings of retrenchment. Sci. Comput. Program. **67**(2), 301–329 (2007)
15. Banach, R., Su, W.: Cyberphysical systems: a behind-the-scenes foundational view. In: Models: Concepts, Theory, Logic, Reasoning and Semantics, pp. 177–201 (2018)

16. Bezanson, J., Edelman, A., Karpinski, S., Shah, V.B.: Julia: a fresh approach to numerical computing. SIAM Rev. **59**(1), 65–98 (2017)
17. Bliudze, S., Furic, S., Sifakis, J., Viel, A.: Rigorous design of cyber-physical systems. Softw. Syst. Model. (SoSyM) **18**(3), 1613–1636 (2019)
18. Blochwitz, T., et al.: The functional mockup interface for tool independent exchange of simulation models. In: Proceedings of the 8th International Modelica Conference, pp. 105–114. Linköping University Press (2011)
19. Börger, E., Stärk, R.: Abstract State Machines: A Method for High-Level System Design and Analysis. Springer, Berlin Heidelberg (2003)
20. Börger, E.: The abstract state machines method for high-level system design and analysis. In: Formal Methods: State of the Art and New Directions, pp. 79–116. Springer (2009)
21. Borutzky, W.: Bond Graph Methodology. Springer (2010)
22. Chua, L.O., Desoer, C.A., Kuh, E.S.: Linear and Nonlinear Circuits. McGraw Hill (1987)
23. David, A., Larsen, K.G., Legay, A., Mikučionis, M., Poulsen, D.B.: Uppaal SMC tutorial. Int. J. Softw. Tools Technol. Transfer **17**(4), 397–415 (2015)
24. De Silva, C.W.: Modeling of Dynamic Systems with Engineering Applications. CRC Press, 2nd edn. (2023)
25. Duindam, V., Macchelli, A., Stramigioli, S., Bruyninckx, H.: Modeling and control of complex physical systems: the port-hamiltonian approach. Springer (2009)
26. Elmqvist, H., Mattsson, S., Otter, M.: Modelica — a language for physical system modeling, visualization and interaction. In: Proceedings of the 1999 IEEE International Symposium on Computer Aided Control System Design (Cat. No.99TH8404), pp. 630–639 (1999)
27. Feynman, R.P., Leighton, R.B., Sands, M.: The Feynman Lectures on Physics: The New, Millennium Basic Books, New York (2010)
28. Henzinger, T.A.: The theory of hybrid automata. In: Proceedings 11th Annual IEEE Symposium on Logic in Computer Science, pp. 278–292. IEEE (1996)
29. Jackson, D., Damon, C.: Elements of style: analyzing a software design feature with a counterexample detector. IEEE Trans. Softw. Eng. **22**(7), 484–495 (1996)
30. Jackson, D.: Software Abstractions: Logic, Language, and Analysis. The MIT Press (2012)
31. Karnopp, D., Margolis, D., Rosenberg, R.: System Dynamics: Modeling, Simulation, and Control of Mechatronic Systems. Wiley, 5th edn. (2012)
32. Kypuros, J.: Dynamics and Control with Bond Graph Modeling. CRC (2013)
33. Leuschel, M., Ishikawa, F. (eds.): Rigorous State-Based Methods: 11th International Conference, ABZ 2025, Düsseldorf, Germany, 10–13 June 2025, Proceedings. Lecture Notes in Computer Science, Springer, Cham (2025)
34. Nipkow, T., Wenzel, M., Paulson, L.C.: Isabelle/HOL: a proof assistant for higher-order logic. Springer (2002)
35. Platzer, A.: Logical Foundations of Cyber-Physical Systems. Springer (2018)
36. Rajkumar, R., Lee, I., Sha, L., Stankovic, J.: Cyber-physical systems: the next computing revolution. Proc. IEEE **98**(1), 136–148 (2010)
37. Rowell, D., Wormley, D.N.: System Dynamics: An Introduction. Prentice Hall (1997)
38. Seshia, S.A.: Explorations in cyber-physical systems education. Commun. ACM **65**(5), 60–69 (2022)
39. Shearer, J.L., Murphy, A.T., Richardson, H.H.: Introduction to System Dynamics. Addison-Wesley (1967)

40. Tiller, M.: Introduction to Physical Modeling with Modelica. Kluwer Academic Publishers, USA (2001)
41. Wellstead, P.: Introduction to Physical System Modelling. Academic Press (1979)